

ActExchanger: Action-grained Collaborative Training for Heterogeneous Embodied AI Agents

Abstract—Multi-agent collaboration has attracted growing attention in embodied artificial intelligence (AI) applications, where a team of agents coordinate their actions to complete a mission, e.g. multiple robotic arms assemble a cabinet in automated manufacturing. Most existing multi-agent reinforcement learning (MARL) approaches exchange knowledge extracted from agents’ local models at each training iteration. While such knowledge exchange improves the mission success rate, it suffers from low learning efficiency due to two main drawbacks. First, independent action optimization across heterogeneous agents creates feature discrepancies, causing transferred knowledge to degrade performance. Second, continuous inter-agent transfers incur severe communication overhead. In this paper, we present ActExchanger, an action-grained MARL solution to extract an action’s specific semantic and spatial features while respecting heterogeneity in agents’ tasks, deployed models, and physical embodiments. ActExchanger guarantees communication-efficient collaborative training by: (i) describing action-grained knowledge with minimal description length; and (ii) exchanging knowledge only for common actions. Extensive comparative evaluations show that ActExchanger achieves a 17.16% improvement in accuracy and a 9.72x reduction in communication cost.

Index Terms—Embodied AI, agent, MARL, action, knowledge

I. INTRODUCTION

Embodied artificial intelligence (AI) applications play a pivotal role in various scenarios such as smart home, healthcare, and automatic manufacturing, where agents perceive, reason, and interact with their surrounding environments [1]. An embodied agent, e.g. a robotic arm [2] or dexterous hand [3], is an integration of AI systems and physical embodiments and it is powered by convolutional neural network (CNN) models, language models or Vision-Language-Action (VLA) models [4]–[8]. Multi-Agent Reinforcement Learning (MARL) [9]–[13] is a dominant technology that allows multiple agents to learn from their environment feedback and collaborate to accomplish a mission [14]. This mission can be decomposed into one task per agent [15] and each **task** consists of a predefined sequence of actions [15]. An **action** lasts for a few seconds and corresponds to multiple *physical movements, rotation, or gripping*. For instance, in Figure 1(a), three robotic arm agents coordinate their actions to complete a mission of cabinet arrangement: agents 1/2 open cabinet doors and agent 3 places objects. Although MARL effectively improves agents’ task success rate in dynamic and unpredictable environments, it has not yet adequately addressed the heterogeneity problem among agents.

Challenges. Existing MARL directly exchanges multiple agents’ **model knowledge** learned from their unique and

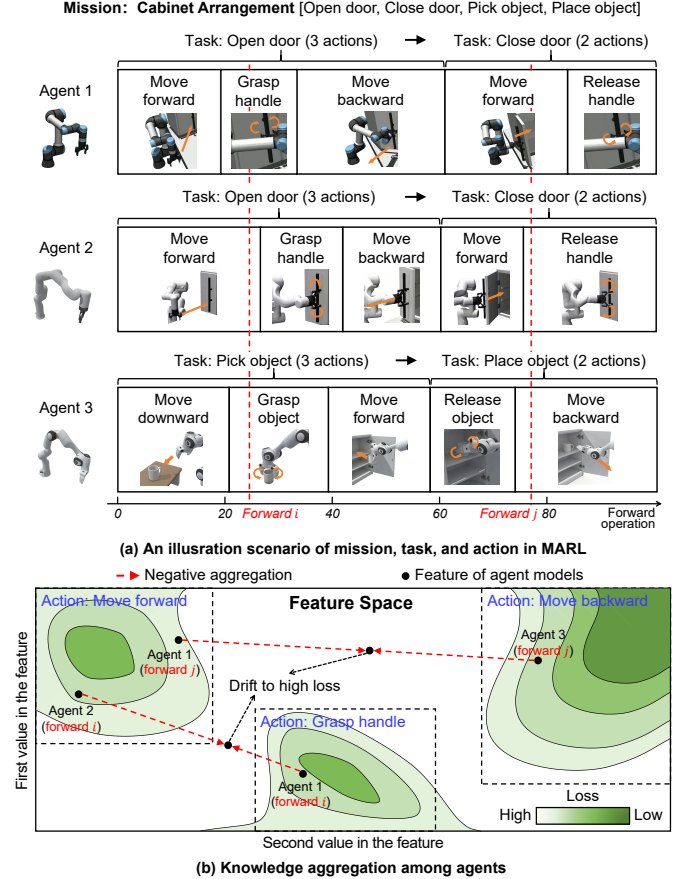


Fig. 1. An example of multi-agent collaboration.

heterogeneous action sequences [10]–[13], with an implicit assumption that these agents are homogeneous or similar. This unfortunately deviates from the real-world application scenarios [16], where each participating agent has its own unique sequence of actions and thus being heterogeneous from each other, arising two major challenges in practice.

Negative knowledge transfer in exchange. MARL methods perform a forward operation of an agent’s local model to output its exchangeable knowledge, whose value depends on the agent’s current action [11]–[13]. As illustrated in Figure 1(a), whether two agents have the same sequence of actions (e.g. agents 1 and 2) or different sequences (e.g. agents 1 and 3), these agents’ outputted knowledge (e.g. forward operation i or j) has a large discrepancy in their feature space. Hence aggregating two agents’ knowledge may drift to a higher loss

space (Figure 1(b)), resulting in negative knowledge transfer (accuracy degradation). The **first** challenge, therefore, requires us to depart from the model-oriented view of knowledge aggregation and manage it at a finer level of action granularity.

Communication constraint refrains heavy communication. In existing approaches, the exchanged knowledge can be local observations [17], model parameters [18], or information summary [19]. Transferring such information per training iteration may incur high communication cost and become infeasible when running on agent devices with limited network bandwidth. The **second** challenge lies in designing a method that maximizes the transferred information of agents' knowledge with low communication cost.

In this paper, we present ActExchanger, a lightweight approach that is designed fundamentally based on the action widely exists in today's embodied AI agents. The key idea of ActExchanger is to introduce an **action-grained knowledge** to represent the semantic and spatial information of an action with minimum description length (MDL) [20]–[22]. This knowledge is a vector of numerical values extracted via forward operation of model parameters. In collaborative training, ActExchanger supports the knowledge transfer between two agents by just exchanging a part of common actions in them and avoids knowledge exchange per training iteration. In doing so, ActExchanger addresses the limitations of existing model-grained MARL techniques in both negative knowledge transfer and high communication cost. In particular, the contributions of this paper are as follows:

- **Action-grained knowledge abstraction.** ActExchanger proposes a novel action-level approach that separates actions from an agent's sub-task and extracts each action's specific knowledge independent of other actions and surrounding environment. This mitigates interference from two agents' heterogeneity in model, physical embodiment, and local observations.
- **Fast collaborative training via action-grained knowledge exchange.** ActExchanger selectively transfers knowledge among peer agents' common actions. Comparing to existing per-iteration exchange mechanism, our approach only communicates knowledge of a few actions existing in both agents, thus improving learning efficiency via positive knowledge transfer and communication cost reduction.
- **Lightweight system implementation and applicability on variety of agents.** ActExchanger implements lightweight systems to support action-grained knowledge extraction and exchange via low-overhead vector and matrix multiplications. Our approach is designed to support prevalent embodied AI agents of different robot arms and dexterous hands.

Summary of experimental results. We extensively compare ActExchanger against state-of-the-art (SOTA) MARL techniques using prevalent embodied AI benchmarks. These benchmarks cover heterogeneous agents, varying numbers of agents, and dynamic environment changes. The comparison

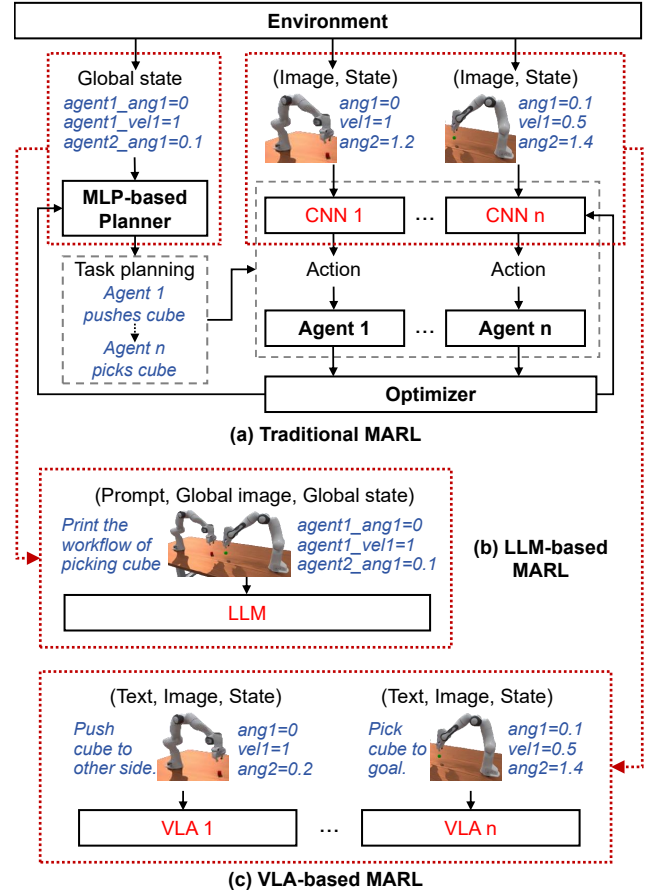


Fig. 2. Development history of MARL techniques

results show: (i) ActExchanger improves accuracy by 17.16% (up to 28.06%) compared to 9 competing MARL baselines under the same resource budget; (ii) ActExchanger saves training time, communication time, transmission volume, and energy consumption by 3.73x, 9.72x, 8.50x, and 4.16x respectively under the same accuracy; and (iii) we comprehensively validate ActExchanger's design choices and discuss its limitations (e.g. maximum supported model sizes) when deployed on resource-constrained agent devices.

II. BACKGROUND OF MULTI-AGENT COLLABORATION

One primary challenge of embodied AI is that agents must be able to continuously learn and adapt to dynamic and evolving environments (e.g. changes in lighting and camera viewpoint) [23]. MARL is a dominant technology to improve agents' task success rate [10], [13]. It allows each agent to continuously interact with the surrounding environment, collect feedback, and update its model parameters [10]. In multi-agent scenarios, peer agents extract their learned knowledge by performing a forward operation on the local models, and exchange the knowledge per/multiple iterations. In the past decade, MARL studies have three stages of development.

CNN-based MARL (Figure 2(a)). At early stages, MARL is designed for CNN-based agents [10]–[13]. Its training

involves three steps. First, a *multilayer perceptron (MLP)-based planner* takes all agents' global states (e.g. joint angles and velocities) as input, and outputs the task planning for each agent. Second, each agent takes its local visual observation, joint angles and velocities as inputs, performs a forward operation on its model, and outputs the corresponding action to interact with the environment. Finally, an *optimizer* uses the environment feedback as input and updates all agents' models to increase their task success rates.

LLM-based MARL (Figure 2(b)). Given the emerging reasoning and in-context learning (ICL) abilities of LLMs [24], LLM-based MARL [25]–[27] replaces CNN-based MARL's planner with a LLM, aiming to improve the precision of task planning. Specifically, the LLM's reasoning ability enables agents to understand the overall mission, identify dependencies among tasks, and generate more accurate task planning. Its ICL ability allows agents to adapt to evolving environments based on prompts.

VLA-based MARL (Figure 2(c)). In recent years, VLA models have become the major driving power of embodied AI agents for two reasons [4], [5]. First, they inherit the reasoning and learning abilities of LLMs because both models have the same transformer architectures. This allows VLA models to replace both CNN models and the MLP-based planners in MARL, while achieving a higher task success rate (e.g. 41.8% higher in the RLbench benchmark [6]). Second, VLA models have the ability to process an additional data modality (text) as agents' task description, thus simultaneously performing task planning and action generation.

III. DESIGN OF ACTEXCHANGER

To enable action-grained collaborative training among agents, two basic research questions need to be answered in ActExchanger: 1) how to define and generate compact action-grained knowledge (Sections III-B and III-C), and 2) in collaborative training, how to efficiently communicate peer agents' knowledge with low communication cost (Sections III-D and III-E).

A. Overview

Figure 3 illustrates the architecture of ActExchanger, which is built upon **action-grained knowledge** (§III-B). This data structure summarizes each action's time-invariant semantic and spatial features using MDL [20]–[22]. In an agent, one knowledge exchange in ActExchanger involves four steps.

At step 1, the **action-grained knowledge generator** (§III-C) encodes the current action a 's local observation and physical embodiment into its action-grained knowledge. This knowledge is a vector of numerical values representing a 's semantic and spatial features.

At steps 2 and 3, the **action-grained knowledge aggregator** (§III-D) first decodes the action-grained knowledge obtained from other agents, and then integrates them with the agent's local model. To support knowledge transfer among heterogeneous agents, the aggregator transforms the local knowledge

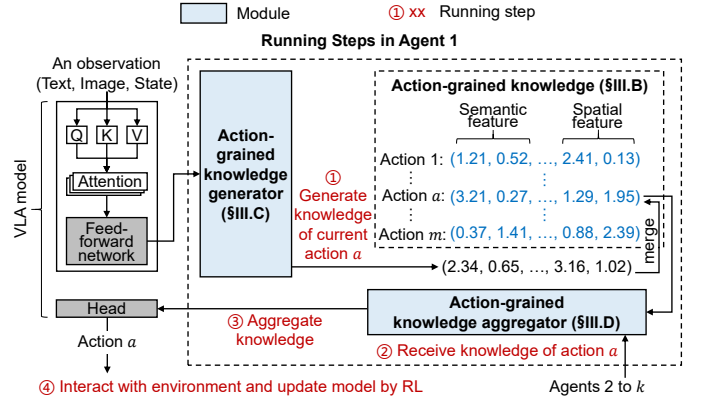


Fig. 3. The architecture of ActExchanger.

and obtained knowledge into unified query, key, and value vectors, and applies an attention operation to aggregate them.

Finally, at step 4, the aggregated knowledge is fed into the model's head layer and transferred into action a . The agent interacts with the environment using this action, obtains feedback, and updates the model by MARL.

B. Action-grained Knowledge

Definition 1: action-grained knowledge is a vector f of $p + q$ numerical values, which can be divided into two parts: (i) *semantic feature* with MDL p : $f^{\text{sem}} = (f_1^{\text{sem}}, \dots, f_p^{\text{sem}})$, and (ii) *spatial feature* with MDL q : $f^{\text{spa}} = (f_1^{\text{spa}}, \dots, f_q^{\text{spa}})$.

Action-grained knowledge is designed with two objectives.

Abstraction of semantic and spatial features. The values in the semantic feature $(f_1^{\text{sem}}, \dots, f_p^{\text{sem}})$ depend on the agent's local observation, and represent the object's affordances (that is, ways to manipulate the object), category, and color. They guide the agent to identify the target object and understand how to manipulate it. In addition, the values in the spatial feature $(f_1^{\text{spa}}, \dots, f_q^{\text{spa}})$ depend on the agent's physical embodiment, and represent the object's position, rotation, and size. They determine the agent's physical movements/rotations on the target object.

Compact knowledge representation. ActExchanger searches the minimum description of f^{sem} and f^{spa} using MDLs in definition 2. Note that this search is only conducted once in the offline pre-training stage. The found MDLs are used to construct action-grained knowledge during MARL training.

Definition 2: MDL of action-grained knowledge is defined as the sum of MDLs of semantic and spatial features. Given the semantic feature f^{sem} and spatial feature f^{spa} , their MDLs p and q are defined as follows [21]:

$$p = L(f^{\text{sem}}); \quad q = L(f^{\text{spa}}) \quad (1)$$

$$L(\mathbf{x}) = \min(\{l \mid I(\mathbf{x}, l) - I(\mathbf{x}, l-1) \leq \alpha \times I(\mathbf{x}, l)\}) \quad (2)$$

$$I(\mathbf{x}, l) \leq \sum_{i=1}^l \text{info}(x_i) \quad (3)$$

$$\text{info}(x_i) = \frac{\sigma(x_i)^2 + \mu(x_i)^2 - 1 - 2 \log \sigma(x_i)}{2} \quad (4)$$

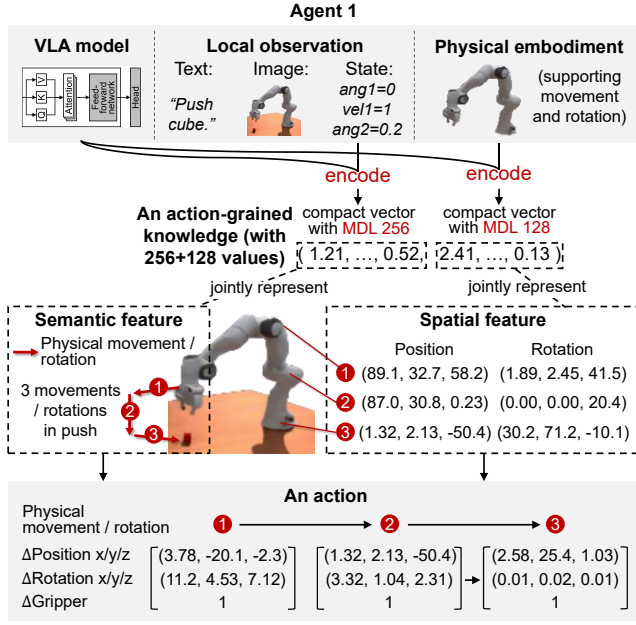


Fig. 4. An example of the action-grained knowledge.

where $L(\mathbf{x})$ is the MDL of vector $\mathbf{x}$ (f^{sem} or f^{spa}), $I(\mathbf{x}, l)$ denotes the amount of information in $\mathbf{x}$ with length l , $\text{info}(x_i)$ denotes the amount of information in x_i (i -th value in $\mathbf{x}$), and $\mu(x_i)$ and $\sigma(x_i)$ denote x_i 's mean and standard deviation.

In Equations (1) and (2), *MDL is the smallest length such that its reduction would cause a relative information loss larger than α (e.g. 5%)*. Equation (3) states that the information contained in l values is upper-bounded by the sum of the information contained in each value. Equation (4) defines the information contained in one value x_i [21].

Search of MDL. This search aims to enumerate vector lengths from large to small, and select the smallest length that does not cause excessive information loss. According to Equations (2) and (1), ActExchanger initializes the semantic/spatial feature's MDL p as a large value (e.g. 1024), and searches for its minimal value using multiple iterations. Each iteration has three steps. Step 1 removes one value f_i^{sem} with the smallest amount of information $\text{info}(f_i^{\text{sem}})$ (Equation (4)), and decreases p by 1. Step 2 calculates the amount of information in f^{sem} before and after the removal, namely $I(f^{\text{sem}}, p)$ and $I(f^{\text{sem}}, p-1)$ (Equation (3)). Finally, step 3 calculates their differences $I(f^{\text{sem}}, p) - I(f^{\text{sem}}, p-1)$. If the difference is significant (e.g. larger than $5\% \times I(f^{\text{sem}}, p)$), it means further decreasing p loses significant semantic features and thus the current p is the minimum one. Otherwise, the search returns to step 1.

Illustration example. Figure 4 shows an example where a robotic arm performs an action of pushing a cube. Its semantic feature indicates that pushing a cube requires three physical movements/rotations. Its spatial feature records key geometric positions and rotations required for three physical movements/rotations. To minimize the description length, ActExchanger uses the VLA model to encode local observation

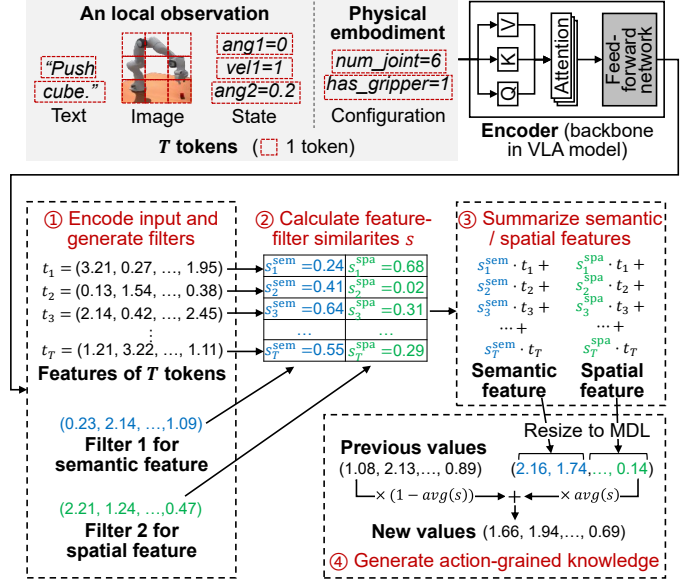


Fig. 5. The running steps in the action-grained knowledge generator.

and physical embodiment into semantic and spatial features, each with an initial length of 1024. ActExchanger then removes values whose information contributions are close to zero, and reduces the lengths of the semantic and spatial features to 256 and 128, respectively.

C. Action-grained Knowledge Generator

Given an agent's VLA model and its current action a , the generator transforms the agent's current local observation and physical embodiment into a 's knowledge. The basic idea is to separately extract semantic and spatial features from each input token, and summarize them to form the action-grained knowledge. As shown in Figure 5, in one forward operation, the generator performs this process using four steps.

Step 1. Input encoding and filter generation. The generator first encodes the input into intermediate feature vectors: the VLA model takes an observation with T tokens as input, and outputs T feature vectors $\{t_1, \dots, t_T\}$ using the last feed-forward network. Each vector t_i stores *all* features extracted from the i -th token.

Subsequently, this step constructs two filters to select semantic and spatial features from t_i . It first prepares a semantic-related text question (e.g. "how to manipulate the cube?") and a spatial-related text question (e.g. "where is the target object on the x/y/z axes?"). These two questions are fed into the VLA model's text encoder, which outputs two vectors g^{sem} and g^{spa} as semantic and spatial filters. These two vectors can serve as filters because the VLA model can align text questions and input tokens in the feature space, and map them into comparable vectors [28]. Hence the filters $g^{\text{sem}} / g^{\text{spa}}$ show higher similarity to a token's feature t_i if this feature contains more semantic/spatial features.

Step 2. Similarity calculation between encoded features and filters. For each token's feature t_i , the generator computes

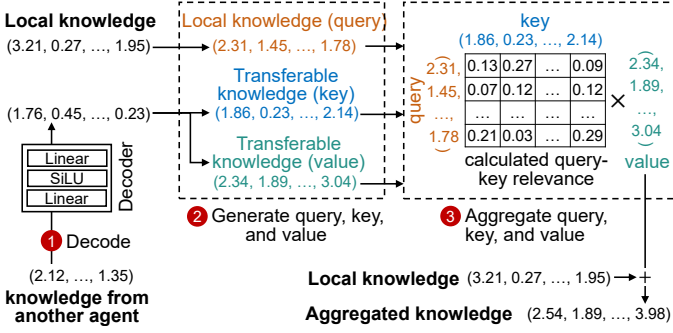


Fig. 6. The running steps in the action-grained knowledge aggregator.

its cosine similarity s_i^{sem} to the semantic filter (Equation (5)) and its cosine similarity s_i^{spa} to the spatial filter (Equation (6)), respectively:

$$s_i^{\text{sem}} = \left(\frac{t_i \cdot g^{\text{sem}}}{\|t_i\| \|g^{\text{sem}}\|} + 1 \right) / 2 \quad (5)$$

$$s_i^{\text{spa}} = \left(\frac{t_i \cdot g^{\text{spa}}}{\|t_i\| \|g^{\text{spa}}\|} + 1 \right) / 2 \quad (6)$$

where $\|\cdot\|$ denotes the Euclidean norm operator.

Step 3. Semantic and spatial feature summarization. The generator obtains the summarized semantic feature by calculating a s_i^{sem} -weighted average of all tokens' features $\{t_1, \dots, t_T\}$. Similarly, it obtains the summarized spatial feature by calculating a s_i^{spa} -weighted average of all tokens' features $\{t_1, \dots, t_T\}$. In this way, the token's feature t_i containing more semantic/spatial features contributes more to the summarized semantic/spatial feature.

Step 4. Action-grained knowledge generation and storage. As shown in Figure 5, the generator first resizes the summarized semantic and spatial features to their MDLs, by removing values with the least information (Equation (4)). The generator then concatenates them as one new action-grained knowledge f' , and stores f' in two ways: (i) the generator directly stores f' if the current action has no previously stored action-grained knowledge; (ii) otherwise, the generator *merges* f' into the previously stored one. This is because features generated at different moments are similar for the same action. Hence storing similar features causes extra overhead.

To merge similar features, the generator first estimates the amount of semantic and spatial features in the new feature f' . This amount is measured by the average similarity $\text{avg}(s)$ between all tokens and two filters (Equation (7)). It then merges f' with the stored/previous feature f by weighting f' with $\text{avg}(s)$ and f with $1 - \text{avg}(s)$ (Equation (8)):

$$\text{avg}(s) = \frac{1}{2T} \sum_{i=1}^T (s_i^{\text{sem}} + s_i^{\text{spa}}) \quad (7)$$

$$f \leftarrow \text{avg}(s) \cdot f' + (1 - \text{avg}(s)) \cdot f \quad (8)$$

D. Action-grained Knowledge Aggregator

In each forward operation, the aggregator combines the local action-grained knowledge with the obtained action-grained

knowledge from other agents. Figure 6 shows the three aggregation steps.

Step 1. Obtained knowledge decoding. The aggregator decodes the obtained action-grained knowledge using a tiny neural network. This network contains two linear layers to resize the knowledge and one activation layer to recover the encoded semantic and spatial features. It outputs the decoded obtained knowledge f_{remote} , which has the same length as the local knowledge f .

Step 2. Query, key, and value generation. The aggregator first maps the local knowledge f into a *query* vector v^{query} , and then maps the decoded knowledge f_{remote} into a *key* vector v^{key} and a *value* vector v^{value} . Specifically, vector v^{key} is used to calculate the relevance between the local knowledge and the obtained knowledge. Vector v^{value} represents the transferable knowledge extracted from the obtained knowledge. Given that both the relevance and the transferable knowledge depend on various factors such as model architecture, local observation, and feature distribution, all mappings are implemented by three learned matrices W_{query} , W_{key} , and W_{value} rather than fixed formulas:

$$v^{\text{query}} = W_{\text{query}} \times f$$

$$v^{\text{key}} = W_{\text{key}} \times f_{\text{remote}}$$

$$v^{\text{value}} = W_{\text{value}} \times f_{\text{remote}}$$

where the values of W_{query} , W_{key} , and W_{value} are obtained through gradient descent in the pre-training.

Step 3. Query, key, and value aggregation. The aggregator performs attention computation [29] on v^{query} , v^{key} , and v^{value} to obtain the aggregated knowledge f_{agg} . It first computes the query-key relevance matrix R :

$$R = \text{row_softmax}((v^{\text{query}})^T \times v^{\text{key}})$$

where $\text{row_softmax}(\cdot)$ denotes row-wise softmax normalization and $(\cdot)^T$ denotes the matrix transpose operation. Each element in R measures the relevance between one value in the local knowledge and one value in the obtained knowledge. The aggregator then applies R to the transferable knowledge v^{value} and combines the result with the local knowledge f :

$$f_{\text{agg}} = R \times (v^{\text{value}})^T + f$$

Hence more relevant values in the transferable knowledge contribute more to the aggregated action-grained knowledge f_{agg} . The aggregated knowledge f_{agg} is fed into the model's last layer and converted into an output action.

E. Communication Overhead Analysis

We analyze the communication overhead between two agents: one knowledge *sender* and one knowledge *receiver*. Let p and q denote the MDLs of semantic and spatial features generated by the sender, and p' and q' denote the length of original semantic and spatial features. Let A^{send} and A^{recv} denote the set of actions in the sender and receiver.

Overhead of each communication. The baseline method transmits one *original* feature with an overhead of $p' + q'$.

In ActExchanger, the sender transmits one action-grained knowledge with *MDL*, incurring the overhead of $p + q$ (i.e. $\frac{p'+q'}{p+q}$ reduction).

Communication frequency. The baseline method triggers one communication per forward operation, i.e. the frequency is 1. In ActExchanger, the sender only triggers one communication when it performs one of the common actions with the receiver. Hence ActExchanger’s communication frequency is $\frac{|A^{\text{send}} \cap A^{\text{recv}}|}{|A^{\text{send}}|}$, where $|\cdot|$ denotes the set size.

Compared to the baseline method, ActExchanger saves the communication overhead by a factor of:

$$\frac{p' + q'}{p + q} \times \frac{|A^{\text{send}}|}{|A^{\text{send}} \cap A^{\text{recv}}|}$$

Taking the workload *cucumber placing* (§IV-A) as an example, where $p = 256$, $q = 128$, $p' = 1024$, $q' = 1024$, $|A^{\text{send}}| = 5$, and $|A^{\text{send}} \cap A^{\text{recv}}| = 3$, ActExchanger saves the communication overhead by 8.89x.

IV. EVALUATION

A. Experimental Settings

Testbeds. We select three commodity computing platforms for embodied AI agents: (1) NVIDIA AGX Xavier with 32GB memory and 512 Volta GPU cores; (2) NVIDIA AGX Orin with 32GB memory and 1792 Ampere GPU cores; and (3) NVIDIA AGX Orin with 64GB memory and 2048 Ampere GPU cores. All platforms are equipped with Ubuntu 22.04 and PyTorch 2.4.0.

Implementation of ActExchanger. We develop three categories of interfaces to decouple ActExchanger’s implementation from workload settings. To adapt ActExchanger to a new workload, developers can implement these interfaces with 100 to 200 LoCs. (1) *Agent*: this interface provides APIs for loading agents from Universal Robot Description Format (`load_agent()`) and defining their mission, tasks, and actions (`define_agent()`); (2) *MARL algorithm*: this interface provides APIs for interacting with environments (`interact()`) and updating models (`update()`); (3) *VLA model*: this interface provides APIs for performing forward/backward operations (`forward()` and `backward()`).

The vector and matrix multiplications occupy the major computational time in ActExchanger’s modules. In implementation, ActExchanger batches multiplications within a short time window into a single CUDA kernel, reducing kernel launch overhead and improving GPU utilization. In addition, multiple agents’ knowledge exchange can be parallelized.

Workloads. We choose four workloads, which represent four typical embodied AI application scenarios.

- *Object picking and placing.* This workload involves two Franka Panda robotic arms, each with seven joints and one gripper. Their mission is to collaboratively move the target object (e.g. cube and sphere) to the goal position. They use the VLA models VLA-Adapter [4] and SmolVLA [30] to output actions, respectively. We develop this workload in the ManiSkill [31] benchmark.

- *Object stacking.* This workload involves one Franka Panda robotic arm and one UR10e robotic arm (with six joints and one gripper). Their mission is to first place a cube at the goal position and stack another object (e.g. sphere) on top of the cube. They use the VLA models VLA-Adapter and OpenVLA [5] to output actions, respectively. We develop this workload in the CoEnv [32] benchmark.
- *Cucumber placing.* This workload involves one Franka Panda robotic arm and two WidowX AI arms (each with six joints and one gripper). Their mission is to collaboratively place the cucumbers into the pot and then close the lid. They use the VLA models VLA-Adapter, SmolVLA, and EfficientVLA [33] to output actions, respectively. We develop this workload in the CoEnv [32] benchmark.
- *Cylinder transfer.* This workload involves five heterogeneous embodied agents: a Franka Panda robotic arm, a WidowX AI robotic arm, a SO100 robotic arm (with five joints and one gripper), an XArm6 robotic arm (with six joints and one gripper), and an Inspire dexterous hand (with five fingers and twelve joints). Their mission is to pass a cylinder through four robotic arms sequentially and place it on the dexterous hand. They use the VLA models VLA-Adapter, SmolVLA, EfficientVLA, TinyVLA [7], and FLOWER [34] to output actions, respectively. We develop this workload in the ManiSkill [31] benchmark.

Each workload runs for 300 minutes and includes 10 different environments that appear sequentially. An **environment** is defined as a unique combination of the target object’s characteristics (e.g. category, shape, and size), lighting conditions (e.g. brightness, color temperature, and light direction), and camera viewpoints [35].

Metric. All workloads use the *success rate of the overall mission* as the accuracy metric.

Baselines. We compare our method with three types of nine MARL techniques:

- *CNN-based MARL*: (1) MAPPO [10] extends the single-agent RL method Proximal Policy Optimization (PPO) to the multi-agent scenarios; (2) DICC [11] models inter-agent dependencies and communication structures through implicit coordination graphs; (3) MAT [12] uses a transformer to model inter-agent information interaction and joint decision-making; and (4) TGCNet [13] uses dynamic directed graph to select how information is communicated among agents.
- *LLM-based MARL*: (1) MAPoRL [25] optimizes the high-level LLM planner through multi-agent post co-training; (2) MAGRPO [26] optimizes the LLM planner through group-relative policy optimization; and (3) MPDF [27] improves the planning precision through meta-policy collaboration and deliberation.
- *VLA-based MARL*: (1) MAPLE [36] improves learning efficiency by predicting future multi-agent states and joint behaviors; and (2) CoMaTrack [37] improves learning efficiency through game-theoretic multi-agent decision training.

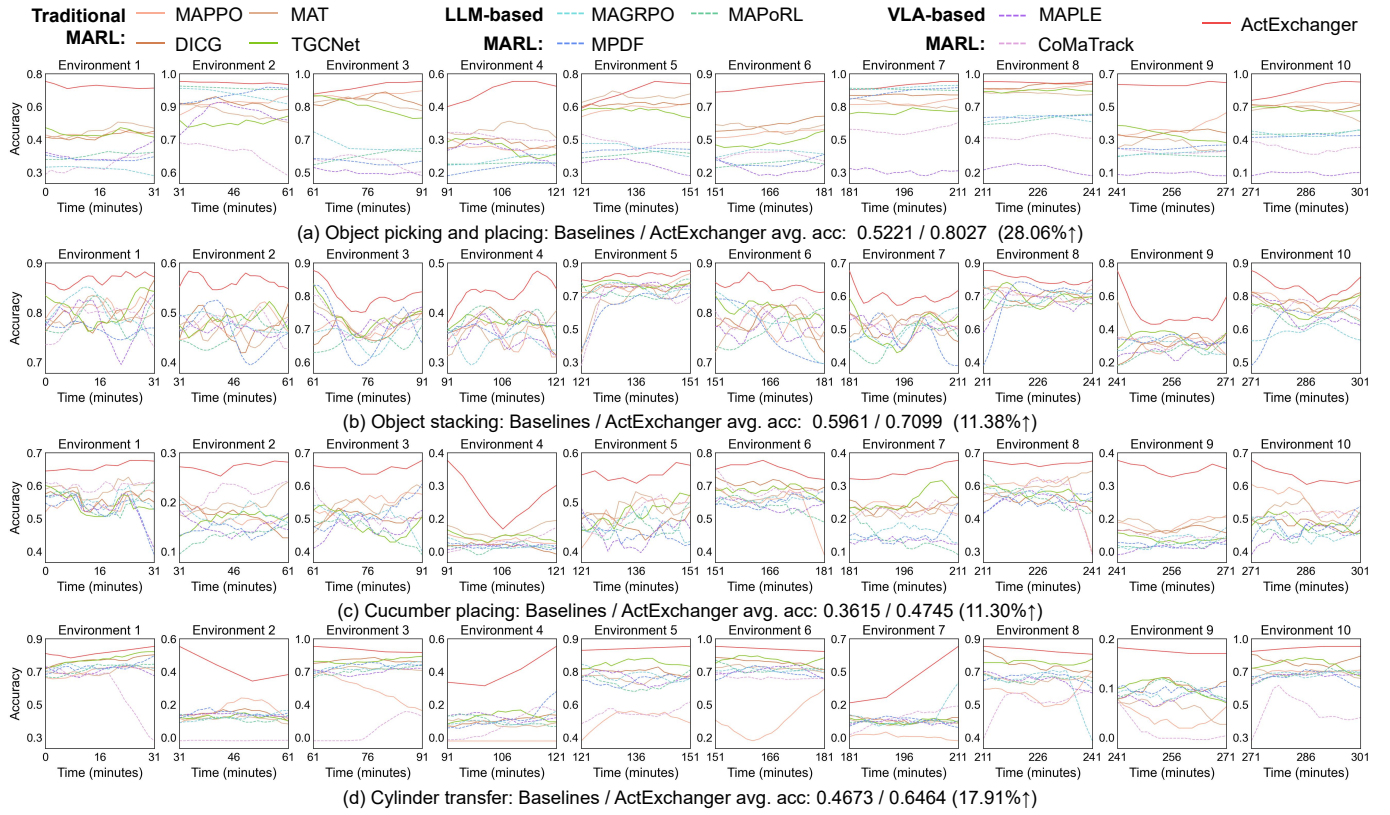


Fig. 7. Accuracy comparison under environment changes.

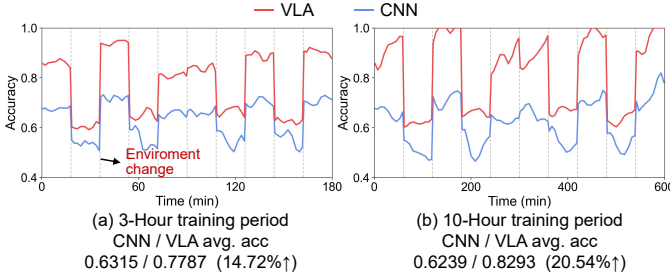


Fig. 8. Accuracy comparison between VLA models and CNNs under environment changes.

B. Comparison of Accuracy under Dynamic Environment

In this evaluation, we compare the accuracy of ActExchanger and baseline techniques under the same resource budget.

Settings. In the offline stage, we pre-train the required VLA models for each workload on an NVIDIA A100 GPU with 80 GB VRAM. The pre-training time ranges from 15.3 to 41.9 hours for the baselines and from 12.7 to 34.6 hours for ActExchanger. Baseline techniques require longer pre-training time for preparing their additional components (e.g. the transformer in MAT). The pre-training is performed once for all methods.

In evaluation, we run four workloads on different com-

puting platforms: (i) two agents in the object picking and placing workload run on two NVIDIA AGX Xavier (32GB), respectively; (ii) two agents in the object stacking workload run on two NVIDIA AGX Orin (32GB), respectively; (iii) three agents in the cucumber placing workload run on one NVIDIA AGX Xavier (32GB) and two NVIDIA AGX Orin, respectively; and (iv) five agents in the cylinder transfer workload run on two NVIDIA AGX Xavier (32GB), two NVIDIA AGX Orin, and one NVIDIA AGX Orin (64GB), respectively.

Results. Figure 7 demonstrates each method’s accuracy in four workloads. We have two key observations. First, ActExchanger achieves the highest accuracy across all workloads and environments. Second, ActExchanger mitigates negative knowledge transfer and improves accuracy quickly after environment changes. This is because ActExchanger generates the action-grained knowledge independent of agents’ heterogeneity and their surrounding environments, and selectively aggregates knowledge of peer agents’ common actions. Both generation and aggregation improve agents’ learning efficiency via positive knowledge transfer.

In contrast, baseline techniques exhibit lower accuracies, and some of them even show a continuous accuracy drop, such as MAPPO in environment 3 of Figure 7(d). This is because the knowledge they generate and exchange depends on the agents’ current action. The heterogeneity between two

TABLE I
OVERHEAD COMPARISON UNDER THE SAME LEARNING ACCURACY. "(A)" TO "(D)" REPRESENT FOUR WORKLOADS, RESPECTIVELY.

	Time (h)				Communication time (min)				Transmission volume (GB)				Energy consumption (10^6 J)				Ratios of common action knowledge in exchange (%)			
	(a)	(b)	(c)	(d)	(a)	(b)	(c)	(d)	(a)	(b)	(c)	(d)	(a)	(b)	(c)	(d)	(a)	(b)	(c)	(d)
MAPPO	1.82	1.47	2.04	2.78	0.95	0.85	2.00	12.96	0.57	0.56	1.17	9.44	0.65	0.66	1.05	1.89	15.09	63.53	43.44	70.43
MAT	2.08	1.65	1.82	2.46	1.15	0.55	2.92	11.94	0.76	0.37	1.86	8.37	0.67	0.72	1.18	1.52	15.78	65.41	35.15	65.26
DICG	1.61	1.29	1.04	2.39	<u>0.22</u>	<u>0.20</u>	<u>0.78</u>	5.72	<u>0.15</u>	<u>0.13</u>	<u>0.49</u>	3.80	0.50	0.53	0.75	1.07	16.29	65.31	37.56	69.55
TGCNet	1.04	1.32	1.22	1.93	0.59	0.71	1.63	<u>5.15</u>	0.38	0.48	0.94	3.25	0.33	0.48	0.52	0.90	16.75	65.32	33.16	60.88
MAGRPO	3.23	2.89	3.03	4.24	1.47	1.39	3.53	37.13	0.97	0.90	2.11	25.24	0.84	1.46	1.66	2.62	16.92	66.56	45.01	<u>72.06</u>
MAPoRL	3.11	3.52	2.25	3.85	1.45	1.48	2.36	40.80	0.91	0.99	1.48	28.32	0.78	1.31	1.16	2.18	16.96	<u>68.96</u>	38.33	70.66
MPDF	2.39	2.17	2.53	3.61	1.37	1.25	2.74	29.82	0.85	0.81	1.76	20.16	0.81	0.88	1.33	2.59	16.73	67.31	48.99	71.97
MAPLE	1.38	1.16	<u>0.95</u>	2.19	0.83	0.69	0.84	17.31	0.49	0.47	0.51	11.58	0.43	0.40	<u>0.51</u>	1.19	<u>17.71</u>	67.87	43.52	70.67
CoMaTrack	1.29	<u>1.07</u>	1.19	<u>1.88</u>	0.65	0.54	0.93	12.28	0.41	0.35	0.59	8.06	0.39	<u>0.36</u>	0.64	1.04	13.49	65.68	43.58	68.09
ActExchanger	0.48	0.39	0.58	0.94	0.08	0.08	0.23	2.54	0.06	0.07	0.15	1.70	0.16	0.14	0.24	0.49	100.00	100.00	100.00	100.00

agents means their current actions, local models, physical embodiment and feedback from the environment usually have a large discrepancy. Their aggregated knowledge may drift to a higher loss space, resulting in training convergency delay and accuracy decrease.

In conclusion, ActExchanger improves accuracy by an average of 17.16% (up to 28.06%) under environment changes.

C. Comparison of Computational and Communication Costs

In this evaluation, we compare the computational and communication costs of ActExchanger and the baseline techniques under the same accuracy.

Settings. Our comparison involves two steps. The first step records the cost of ActExchanger when it achieves its highest accuracy in each environment. In the same environment, the second step continues training each baseline until it reaches the same accuracy as ActExchanger, and records its cost.

We report four overhead metrics: (i) training time (hours), (ii) communication time (min), (iii) transmission volume (GB), and (iv) energy consumption (10^6 J). Communication time is measured under the network bandwidth 100Mbps. Energy consumption is measured using an REMD-10AL power monitor. We also report one additional auxiliary metric: ratio of common actions in knowledge exchange (%). This ratio denotes the volume of exchanged knowledge of common actions divided by the volume of all exchanged knowledge. A larger ratio indicates a larger proportion of effective communication, namely a higher probability of positive knowledge transfer.

Results. Table I reports the measured metrics. We can observe that ActExchanger has the shortest model training time, communication time, transmission volume, and energy consumption. This is because our approach encodes knowledge with MDL and only exchanges knowledge when two agents perform the same action. That is, it has the highest ratios of common actions in knowledge exchange, achieving the largest probability of positive knowledge transfer.

In contrast, baseline methods cause large communication costs, because they frequently exchange knowledge with the original length in each forward operation. In particular, LLM-based MARL methods incur 3.03x higher communication time and 3.06x larger transmission volume than other baselines.

This is because they transfer multi-modal inputs (e.g. historical images and prompts) between the LLM and agents for more precise task planning.

ActExchanger’s training time breakdown. We measure the wall-clock time of ActExchanger’s two modules. Specifically, we employ CUDA timers `torch.cuda.Event()` to measure the GPU clock time between the start and end of each module. Figure 9 illustrates the results in all workloads. We can see ActExchanger’s modules complete within 1s, i.e., 3% of the time of one training iteration (namely one batch of sampling, environment interaction, and training). This is because both modules in our approach only perform several vector and matrix multiplications at one training iteration. These multiplications can be executed in parallel using CUDA kernels.

In conclusion, ActExchanger reduces training time, communication time, transmission volume, and energy consumption by an average of 3.73x, 9.72x, 8.50x, and 4.16x respectively under the same accuracy.

D. Design Choice Validation by Ablation

Following the settings of Section IV-B, we evaluate ActExchanger’s design choices using the object picking and placing workload.

Length of action-grained knowledge. Figure 10(a) shows that the action-grained knowledge with MDL brings 5.02% higher accuracy than the one with original length. This is because this knowledge length filters out action-irrelevant features such as illumination, while keeping an action’s semantic and spatial information.

Generation of action-grained knowledge. Figure 10(b) shows that generating action-grained knowledge using both semantic and spatial features achieves 15.13% higher accuracy than the solutions using individual semantic/spatial features or unprocessed intermediate features.

Time-invariant action-grained knowledge. For an action, ActExchanger merges all historical and newly generated knowledge into one vector. A straightforward alternative is to keep each knowledge in a separate vector to retain temporal differences. Figure 10(c) shows that ActExchanger achieves similar accuracy to this alternative. This result indicates that

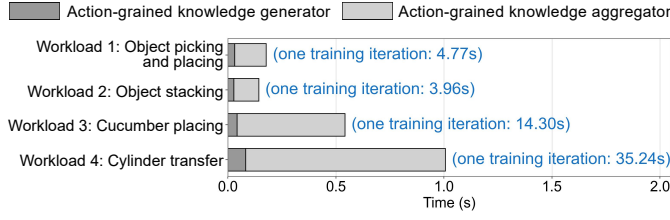


Fig. 9. Training time breakdown of ActExchanger’s modules.

the same action’s knowledge keeps stable over time, and hence can be merged into one vector to reduce communication cost.

Aggregation of action-grained knowledge. Figure 10(d) shows that aggregating action-grained knowledge generated for the same action achieves 25.97% higher accuracy than three optional solutions: (i) no aggregation; (ii) aggregating knowledge in the current forward operation; and (iii) aggregating knowledge in a random forward operation.

E. Discussions and Limitations

ICL ability. We select a latest ICL method for VLA models, RICL [38], and compare its accuracy with ActExchanger on the object picking and placing workload. The results show that RICL achieves 35.43% lower accuracy than ActExchanger. It indicates that current VLA models lack sufficient ICL abilities, and still require resource-intensive reinforcement learning to adapt to evolving environments.

Comparison between VLA models and CNNs. To verify VLA models’ advanced abilities, we test the object picking and placing workload by replacing VLA-Adapter with a representative CNN (EfficientNet [39]), and compare the learning accuracies of both models. The result in Figure 8 shows that the CNN decreases learning accuracy by 14.72% and 20.54% when the training periods are 3 and 10 hours, respectively.

Maximum supported model size. Taking VLA-Adapter as an example, our evaluation shows that ActExchanger can train VLA models as large as 11.3 GB, 11.3 GB, and 22.4 GB on NVIDIA AGX Xavier (32 GB), NVIDIA AGX Orin, and NVIDIA AGX Orin (64 GB), respectively. This is because the action-grained knowledge with MDL occupies negligible memory footprint (2 MB to 43 MB on four workloads). ActExchanger’s generation and aggregation modules also have small memory overhead (25 MB to 130 MB on four workloads).

V. RELATED WORK

CNN-based MARL. Early *non-hierarchical* MARL methods [10], [12], [13] directly learn from shared rewards without explicit planning, thus often struggling with complex missions. To address this issue, *hierarchical MARL* [40], [41] first decomposes a complex mission into a plan of multiple tasks, and learns them separately. For example, FMH [40] and MS-MARL [41] design a manager agent to decompose the mission for worker agents. PSMA [42] decomposes the mission from potential fields and intrinsic rewards.

LLM-Based MARL. Some studies leverage LLMs’s reasoning ability for task planning [24]. For example, ROCO [43]

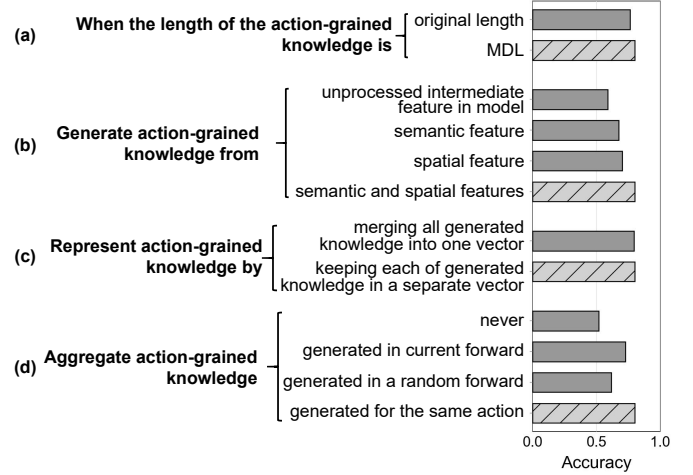


Fig. 10. Accuracies of ActExchanger under different design choices.

and CoELA [44] generate task planning through multi-round dialogue and chain-of-thought reasoning. When encountering evolving environments, they maintain planning accuracy by optimizing LLMs’ prompts [45], memories [46], and collaboration topologies [27].

VLA-Based MARL. With larger model sizes and multi-modal inputs, VLA models can improve agents’ environmental perception and action generation abilities. For example, CoMaTrack [37] and MAPLE [36] replace the CNNs in non-hierarchical and hierarchical MARL with Qwen2.5-VL-based VLA models, improving task rate by nearly 50%.

Relationship to ICL. ICL techniques [24] adapt agents to evolving environments by adjusting their non-parametric contexts such as prompts [47], memory, and tools [48]. Its effectiveness relies on the assumption that the pre-trained model already contains generalizable knowledge, hence specific contexts can guide the agent to retrieve knowledge relevant to each new environment. However, current VLA models have insufficient ICL ability due to scarce pre-training data [38], which needs costly human demonstrations to label paired text, image, and actions [5].

VI. CONCLUSION

This paper presents the design, implementation, and evaluation of ActExchanger, a collaborative model training approach for heterogeneous embodied AI agents. ActExchanger generates action-grained knowledge with MDL in spite of agents’ heterogeneity, while selectively exchanging such knowledge among agents to improve learning efficiency with low communication overhead. We conduct extensive evaluation on realistic embodied AI workloads against the latest MARL methods. The results demonstrate the efficacy and practicality of ActExchanger across different robot arms and dexterous hands.

REFERENCES

- [1] P. Fung, Y. Bachrach, A. Celikyilmaz, K. Chaudhuri, D. Chen, W. Chung, E. Dupoux, H. Jégou, A. Lazaric, A. Majumdar, A. Madotto,

- F. Meier, F. Metze, T. Moutakanni, J. Pino, B. Terver, J. Tighe, and J. Malik, "Embodied AI agents: Modeling the world," *CoRR*, 2025.
- [2] S. Haddadin, "The franka emika robot: a standard platform in robotics research [survey]," *IEEE Robotics & Automation Magazine*, vol. 31, no. 4, pp. 136–148, 2024.
- [3] Y. Zhong, X. Huang, R. Li, C. Zhang, Z. Chen, T. Guan, F. Zeng, K. N. Lui, Y. Ye, Y. Liang *et al.*, "Dexgraspvla: A vision-language-action framework towards general dexterous grasping," in *AAAI'26*, vol. 40, no. 22, 2026, pp. 18 836–18 844.
- [4] Y. Wang, P. Ding, L. Li, C. Cui, Z. Ge, X. Tong, W. Song, H. Zhao, W. Zhao, P. Hou, S. Huang, Y. Tang, W. Wang, R. Zhang, J. Liu, and D. Wang, "Vla-adapter: An effective paradigm for tiny-scale vision-language-action model," in *AAAI'26*, 2026, pp. 18 638–18 646.
- [5] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "Openvla: An open-source vision-language-action model," in *CoRL'25*, 2024, pp. 2679–2713.
- [6] P. Li, Y. Chen, H. Wu, X. Ma, X. Wu, Y. Huang, L. Wang, T. Kong, and T. Tan, "Bridgevla: Input-output alignment for efficient 3d manipulation learning with vision-language models," *NeurIPS'26*, pp. 63 635–63 673, 2026.
- [7] J. Wen, Y. Zhu, J. Li, M. Zhu, Z. Tang, K. Wu, Z. Xu, N. Liu, R. Cheng, C. Shen, Y. Peng, F. Feng, and J. Tang, "Tinyvla: Toward fast, data-efficient vision-language-action models for robotic manipulation," *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3988–3995, 2025.
- [8] T. Majima and T. Nishio, "Loss-robust split vla inference framework toward real-time cloud-controlled robots: Proof of concept," in *INFOCOM'26*, 2026, pp. 2995–2996.
- [9] P. Dai and L. Duan, "Global convergence for multi-agent reinforcement learning in unreliable communication networks," in *INFOCOM'26*, 2026, pp. 1–10.
- [10] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. M. Bayen, and Y. Wu, "The surprising effectiveness of PPO in cooperative multi-agent games," in *NeurIPS'22*, 2022.
- [11] S. Li, J. K. Gupta, P. Morales, R. E. Allen, and M. J. Kochenderfer, "Deep implicit coordination graphs for multi-agent reinforcement learning," in *AAMAS'21*, 2021, pp. 764–772.
- [12] M. Wen, J. G. Kuba, R. Lin, W. Zhang, Y. Wen, J. Wang, and Y. Yang, "Multi-agent reinforcement learning is a sequence modeling problem," in *NeurIPS'22*, 2022.
- [13] Z. Zhang, B. He, B. Cheng, and G. Li, "Bridging training and execution via dynamic directed graph-based communication in cooperative multi-agent systems," in *AAAI'25*, 2025, pp. 23 395–23 403.
- [14] Y. Liu, W. Chen, Y. Bai, G. Li, W. Gao, and L. Lin, "Aligning cyber space with physical world: A comprehensive survey on embodied AI," *CoRR*, 2024.
- [15] Z. Wang, Y. Chen, Y. Liu, J. Ye, P. Chen, C. Lu, S. Liu, B. Yu, and J. Jia, "Vp-vla: Visual prompting as an interface for vision-language-action models," *CoRR*, 2026.
- [16] X. Liu, D. Guo, X. Zhang, and H. Liu, "Heterogeneous embodied multi-agent collaboration," *IEEE Robotics and Automation Letters*, vol. 9, no. 6, pp. 5377–5384, 2024.
- [17] O. Kilinc and G. Montana, "Multi-agent deep reinforcement learning with extremely noisy observations," *CoRR*, 2018.
- [18] Y. Li, S. He, Y. Li, Y. Shi, and Z. Zeng, "Federated multiagent deep reinforcement learning approach via physics-informed reward for multi-microgrid energy management," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 5, pp. 5902–5914, 2023.
- [19] S. Sukhbaatar, A. Szlam, and R. Fergus, "Learning multiagent communication with backpropagation," in *NeurIPS'16*, 2016, pp. 2244–2252.
- [20] P. D. Grünwald, *The minimum description length principle*. MIT press, 2007.
- [21] J. Chai, Y. Xiao, and G. Shi, "On the rate-distortion-complexity tradeoff for semantic communication," *IEEE Internet of Things Journal*, 2026.
- [22] M. Zhou, Y. Liu, W. Nie, L. Xie, and Z. Tian, "Secure mobile crowdsourcing for wlan indoor localization," in *INFOCOM'18 (Workshops)*, 2018, pp. 480–485.
- [23] Y. Liu, H. Li, S. Tian, Y. Qin, Y. Chen, Y. Zheng, Y. Huang, and D. Zhao, "Towards long-lived robots: Continual learning vla models via reinforcement fine-tuning," *CoRR*, 2026.
- [24] L. Berti, F. Giorgi, and G. Kasneci, "Emergent abilities in large language models: A survey," *CoRR*, 2025.
- [25] C. Park, S. Han, X. Guo, A. E. Ozdaglar, K. Zhang, and J. Kim, "Maporl: Multi-agent post-co-training for collaborative large language models with reinforcement learning," in *ACL'25*, 2025, pp. 30 215–30 248.
- [26] S. Liu, Z. Liang, X. Lyu, and C. Amato, "LLM collaboration with multi-agent reinforcement learning," in *AAAI'26*, 2026, pp. 32 150–32 158.
- [27] W. Yang and J. Thomason, "Learning to deliberate: Meta-policy collaboration for agentic llms with multi-agent reinforcement learning," in *AAAI'26*, 2026, pp. 29 820–29 828.
- [28] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, " π_0 : A vision-language-action flow model for general robot control," *CoRR*, 2024.
- [29] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, "Attention is all you need," *NeurIPS'17*.
- [30] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadène, "Smolvla: A vision-language-action model for affordable and efficient robotics," *CoRR*, 2025.
- [31] S. Tao, F. Xiang, A. Shukla, Y. Qin, X. Hinrichsen, X. Yuan, C. Bao, X. Lin, Y. Liu, T. Chan, Y. Gao, X. Li, T. Mu, N. Xiao, A. Gurha, Z. Huang, R. Calandra, R. Chen, S. Luo, and H. Su, "Maniskill3: GPU parallelized robotics simulation and rendering for generalizable embodied AI," *CoRR*, 2024.
- [32] L. Kang, Y. Fan, R. Li, H. Zhou, Y. Qin, Z. Zhang, S. Huang, X. Song, Z. Zhang, B. N. Y. Chen, Z. Yin, D. Zhou, W. Zuo, and L. Bai, "Coenv: Driving embodied multi-agent collaboration via compositional environment," *CoRR*, 2026.
- [33] Y. Yang, Y. Wang, Z. Wen, L. Zhongwei, C. Zou, Z. Zhang, C. Wen, and L. Zhang, "Efficientvla: Training-free acceleration and compression for vision-language-action models," *NeurIPS'26*, pp. 40 891–40 914, 2026.
- [34] M. Reuss, H. Zhou, M. Rühle, Ö. E. Yagmurlu, F. Otto, and R. Lioutikov, "FLOWER: democratizing generalist robot policies with efficient vision-language-action flow policies," *CoRR*, 2025.
- [35] R. Garcia, R. Strudel, S. Chen, E. Arlaud, I. Laptev, and C. Schmid, "Robust visual sim-to-real transfer for robotic manipulation," in *IROS'23*, 2023, pp. 992–999.
- [36] R. Yasarla, D. Hegde, H. Cheng, S. Han, Y. Shi, M. Sadeghigooghari, H. Ackermann, L. Liu, P. Desai, F. Porikli, M. Ghavamzadeh, and H. Cai, "MAPLE: latent multi-agent play for end-to-end autonomous driving," *CoRR*, 2026.
- [37] Y. Liu, L. Gao, L. Liu, M. Lv, and Y. Cai, "Comatrack: Competitive multi-agent game-theoretic tracking with vision-language-action models," *CoRR*, 2026.
- [38] K. Sridhar, S. Dutta, D. Jayaraman, and I. Lee, "Ricl: Adding in-context adaptability to pre-trained vision-language-action models," *CoRR*, 2025.
- [39] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *ICML'19*, 2019, pp. 6105–6114.
- [40] S. Ahilan and P. Dayan, "Feudal multi-agent hierarchies for cooperative reinforcement learning," *CoRR*, 2019.
- [41] X. Kong, B. Xin, F. Liu, and Y. Wang, "Revisiting the master-slave architecture in multi-agent deep reinforcement learning," *CoRR*, 2017.
- [42] S. Li, H. Jiang, Y. Liu, J. Zhang, X. Xu, and D. Liu, "Generative subgoal oriented multi-agent reinforcement learning through potential field," *Neural Networks*, vol. 179, p. 106552, 2024.
- [43] Z. Mandi, S. Jain, and S. Song, "Roco: Dialectic multi-robot collaboration with large language models," in *ICRA'24*, 2024, pp. 286–299.
- [44] H. Zhang, W. Du, J. Shan, Q. Zhou, Y. Du, J. B. Tenenbaum, T. Shu, and C. Gan, "Building cooperative embodied agents modularly with large language models," in *ICLR'24*, 2024.
- [45] G. Chen, S. Dong, Y. Shu, G. Zhang, J. Sesay, B. Karlsson, J. Fu, and Y. Shi, "Autoagents: A framework for automatic agent generation," in *IJCAI'24*, 2024, pp. 22–30.
- [46] K. Chen, X. Li, T. Yang, H. Wang, W. Dong, and Y. Gao, "Mdteamgpt: A self-evolving llm-based multi-agent framework for multi-disciplinary team medical consultation," *CoRR*, 2025.
- [47] D. Peng, Y. Zhou, Q. Chen, J. Liu, J. Chen, L. Qin, and W. Che, "Dlpo: Towards a robust, efficient, and generalizable prompt optimization framework from a deep-learning perspective," *CoRR*, 2025.
- [48] M. A. Haque, J. Williams, S. Siddique, M. H. Islam, H. Ali, K. D. Gupta, and R. George, "Advanced tool learning and selection system (atlass): A closed-loop framework using llm," in *SOSE'25*, 2025, pp. 64–73.